INSTITUTE OF TECHNOLOGY OF CAMBODIA

DEPARTMENT : APPLIED MATHEMATIC AND STATISTICS

Department of Applied Mathematics and Statistics

SUBJECT: ADVANCE PROGRAMMING FOR DATA SCIENCE

CHAPTER 3: NAME THAT CAN'T BE MISCONTRUED

Group 13

1. **YOUS Nisaiy**
2. **RY Chhorly**
3. **LUN Rathana**
4. **PHAL Menghak**

Lecturer: VANN Visal

CONTENT

1. Introduction
2. Clip()
3. Inclusive Ranges
4. Boolean Names and Matching Users’ expectations
5. Evaluating Multiple Name Candidates
6. Summary

INTRODUCTION

Miscontrued: means understood wrongly: the reader takes a different meaning from what the writer intended.

why it is important

1. Causes bugs: Ambiguous names lead to misunderstandings (like off-by-one errors).
2. Hard to detect: Code runs and passes tests while behaving incorrectly.
3. Wastes time: Forces readers to stop and check documentation or code.
4. Bad for collaboration: Teammates (or future you) may misinterpret your intent.
5. High ROI: Renaming takes seconds, but debugging takes hours.

FILTER

Filter() is an ambiguous function name because it isn't clear whether it keeps or removes items.

```
results = Database.all_objects.filter("year <= 2011")
```

What does the results contain?

- Object whoose year is <= 2011
- Object whoose year is not <= 2011

SELECT() for to pick out

exclude() for to get rid of

1. Clip(text, length)

1. it can have two meanings:

Remove some characters from the end.
Keep only a certain number of characters.

```
text = "Hello Python"
length = 5
print(text[:-length])
```

2. truncate_text means to shorten the text.

```
text = "Hello Python"
max_chars = 5
truncate_text = text[:max_chars]
print(truncate_text)
```

2. Prefer min and max for Limits

1. Min (minimum): The smallest amount allowed.

2. Max (maximum): The largest amount allowed.

```
MIN_AGE = 18
MAX_AGE = 60
age = 25
print(age >= MIN_AGE and age <= MAX_AGE)
```

RANGES: FIRST/LAST VS BEEGIN /END

Inclusive Ranges(first/last)

⚠ The Problem with Words Like `stop`

When writing range functions, parameter names like `stop` are inherently ambiguous. Does it stop *before* the value or *at* the value?

```
play_songs(start="Jazz", stop="Rock")
```

The Clear Fix: Playlist Example

Use `first` and `last` to signal an inclusive range where endpoints are guaranteed to be included.

```
play_songs(first="Jazz", last="Rock")
```

Inclusive/Exclusive Ranges(Begin/end)

⇄ The 'Half-Open' Concept

A half-open range includes the start point, but stops right *before* the end point.

Why we use it: Avoids awkward math and edge-case timestamps (like trying to calculate midnight down to the millisecond).

Standard choice for array slicing and intervals.

Date Range Example

Printing all events that happened on October 16:

```
PrintEventsInRange(
  begin="OCT 16 12:00am",
  end="OCT 17 12:00am"
)
```

Clean logic without needing 11:59:59.999pm hacks.

Quick Takeaway

Use first/ last for Inclusive Ranges

Perfect when both endpoints must be explicitly processed eliminating ambiguity.

Use Begin / end for half-open Ranges

The standard idiomatic choice in programming for clean slicing and interval logic

NAMING BOOLEANS

A Boolean variable represents one of two values: True or False.

Example:

```
bool read_password = true;
```

The challenge is naming it so that the reader immediately understands what True means.

- We need to read the password
- The password has already been read

In this case, it's best to avoid the word "read," and name it need_password or user_is_authenticated instead.

In general, adding words like **is**, **has**, **can**, or **should** can make booleans more clear.

Prefix	Example	Meaning
is_	is_price_valid	The price is valid
has_	has_missing	The data is missing value
can_	can_predict_price	Prediction is possible
should_	should_retrain_model	Retraining is recommended

Remember: When naming a Boolean, ask, "What does True mean?"

MATCH USERS' EXPECTATIONS

The name of a function should match what programmers normally expect that function to do. Chapter 3 discusses this principle using conventions such as get*() functions and the meaning of a collection's size().

1. **get*() vs. compute*():**
- Expectation: Instant $O(1)$ lightweight field accessor.
- Problem: When it calculates across millions of items dynamically, callers invoke it carelessly in loops.
- Fix: Use computeMean() or calculateMean().

2. **size() vs. countElements():**
- Expectation: Constant-time $O(1)$ size lookup (like vector or map).
- Problem: In pre-C++11 std::list, it iterated element by element ($O(N)$), causing an $O(N^2)$ slowdown.
- Fix: If an operation is expensive, name it with action verbs like countElements().

3. **The "Wizard Mode" Trap:**
- Expectation: User-friendly, guided setup.
- Reality: Raw terminal command prompt ("you are the wizard").
- Lesson: Don't invent clever terms that invert user intuition.

THANK YOU

